The Hegemoney organization has been flagged.
Because of that, your organization is hidden from the public. If you believe this is a mistake, contact s
reviewed.

Gundam-MS / **QCSuper**
forked from P1sec/QCSuper

QCSuper is a tool communicating with Qualcomm-based phones and modems, allowing to
among other things.

Manage topics

48 commits **1** branch **0** packages **0** releases

Branch: **master** New pull request Create new file

This branch is even with P1sec:master.

p1-mmr Merge branch 'master' of https://github.com/P1sec/QCSuper

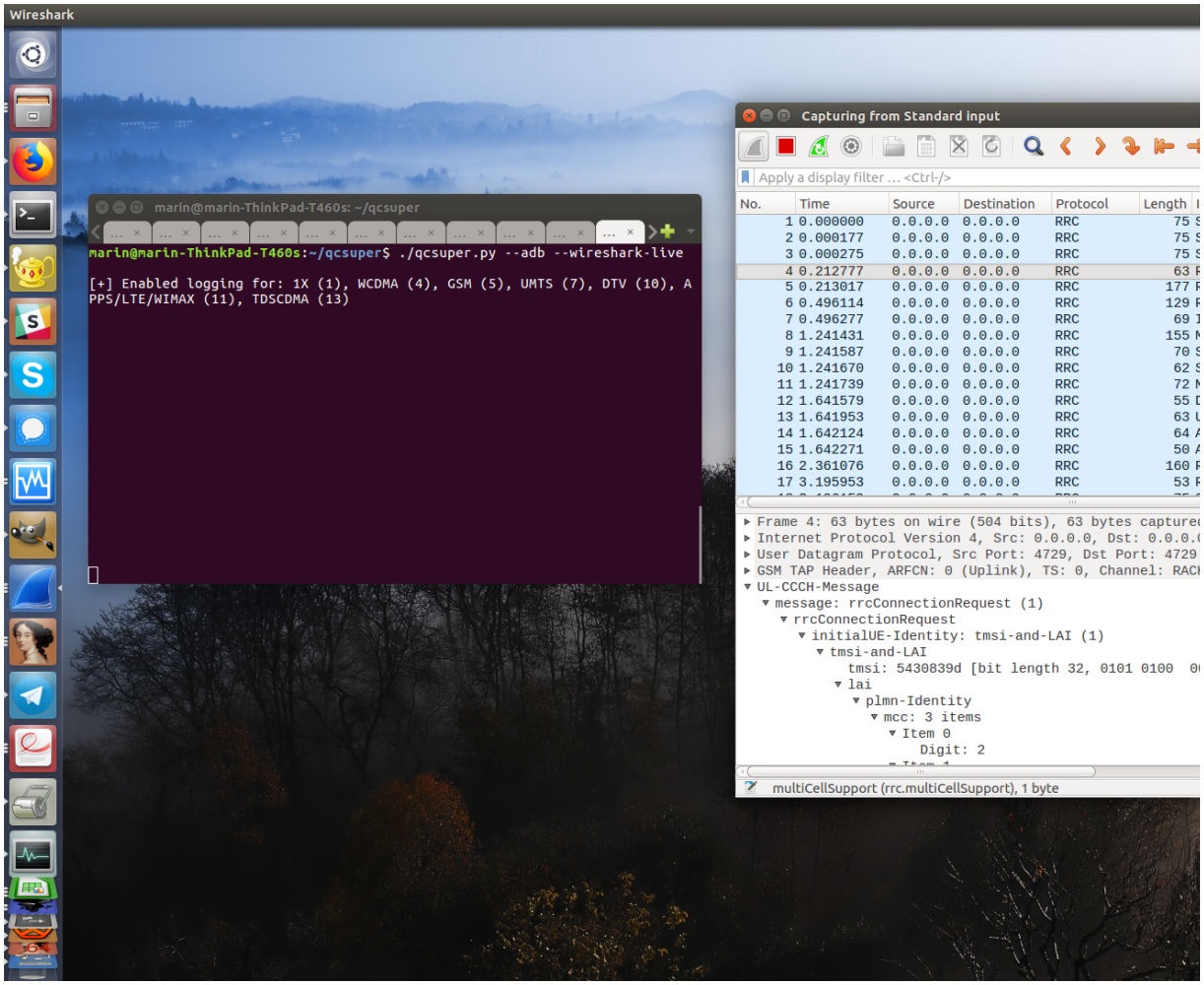
docs	Fix wording in the QCSuper architecture documentation
inputs	Set the standard output to be unbuffered from the "adb_bridge" module
modules	Don't consider that we're appending to a file when writing output to ...
protocol	Add READMEs in the source subdirectories
.gitignore	Initial push to Github
LICENSE	Initial push to Github
README.md	Merge branch 'master' of https://github.com/P1sec/QCSuper
qcsuper.py	Initial push to Github

README.md

QCSuper

QCSuper is a tool communicating with Qualcomm-based phones and modems, allo
frames, among other things.

It will allow you to **generate PCAP** captures of it using either a rooted Android phon
capture in another format.



After having installed it, you can plug your rooted phone in USB and using it is as si

```
./qcsuper.py --adb --wireshark-live
```

It uses the Qualcomm Diag protocol, also called QCDM or DM (Diagnostic Monitor)
phone's baseband.

**You want support, to report that you device works or does not work or you'd like
research on the Diag protocol? You can come talk on IRC (`#qcsuper` at Freenode**

Table of contents

- **Installation**
  - Ubuntu and Debian installation
  - Windows installation
- Supported protocols
- **Usage notice**

Annexes:

- Using QCSuper with an USB modem
- Supported devices
- Related tools using the Diag protocol

Blog post/demo: Presenting QCSuper: a tool for capturing your 2G/3G/4G air traffic

More documentation:

- The Diag protocol
- QCSuper architecture

Installation

QCSuper was tested and developed on Ubuntu 16.04, 18.04 and Windows 7. It depe

To use it, your phone must be rooted or expose a diag service port over USB. In ord your phone, look up the phone's model on a site like GSMArena and check whether

In order to open PCAP files produced by QCSuper, you can use any Wireshark 2.x fo least Wireshark 2.5.x for 4G frames (and 2.6.x for individual NAS messages decrypt currently provides a recent enough build for all versions.

Ubuntu and Debian installation

Open a terminal and type the following:

```
# Download QCSuper
```

```
git clone git@github.com:P1sec/QCSuper.git qcsuper
cd qcsuper

# Install dependencies
sudo apt install python3-pip wireshark
sudo pip3 install --upgrade pyserial crcmod https://github.com/P1sec/pyc
```

Windows installation

On Windows, you will need to download and install your phone's USB drivers from y
way, search for your phone's model + "USB driver" or "ADB driver" on Google for ins

Then, you need to ensure that you can reach your device using `adb`. You can find a
setup `adb` here. The `adb shell` command must display a prompt to continue.

Then, follow these links on order to:

- Install Python 3.6 or more recent (be sure to check options to include it into PA
pip)
- Install Wireshark 2.6 or more recent
- Download and extract QCSuper

To install the required Python modules, open your command prompt and type:

```
pip3 install --upgrade pyserial crcmod https://github.com/P1sec/pycrate/
```

Still in your command prompt, move to the directory containing QCSuper using the
commands (which should start with `qcsuper.py` instead of `./qcsuper.py`).

Supported protocols

QCSuper supports capturing a handful of mobile radio protocols. These protocols a
standard header (encapsulated into UDP/IP) permitting to identify the protocol, and
PCAP file that is fully analyzable using Wireshark.

2G/3G/4G protocols can be broken into a few "layers": layer 1 is about the digital ra
2 handles stuff like fragmentation and acknowledgement, layer 3 is the proper signa

QCSuper allows you most often to capture on layer 3, as it is the most pratical to an
the Diag protocol provides natively (and some interesting information is here).

- 2G (GSM): Layer 3 and upwards (RR/...)
- 2.5G (GPRS and EDGE): Layer 2 and upwards (MAC-RLC/...) for data acknowlec
- 3G (UMTS): Layer 3 and upwards (RRC/...)
  - Additionally, it supports reassembling SIBs (System Information Blocks, the separate GSMTAP frames, as Wireshark currently can't do it itself: flag `--r`
- 4G (LTE): Layer 3 and upwards (RRC/...)
  - Additionally, it supports putting decrypted NAS message, which are embec packet, in additional frames: flag `--decrypt-nas`

By default, the IP traffic sent by your device is not included, you see only the signal traffic you generate using the `--include-ip-traffic` option (IP being barely the la 2G/3G/4G, at the detail that its headers may be compressed (ROHC) and a tiny PPP

The data traffic you send uses a channel different from the signalling traffic, this ch traffic; QCSuper should thus show you all details relevant to how this channel is init

Usage notice

In order to use QCSuper, you specify one input (e.g: `--adb` (Android phone), `--usl` (`--wireshark-live` for opening Wireshark, `--pcap-dump` for writing traffic to a PC information about the device...).

A few commands you can type are:

```
# Open Wireshark directly, using a rooted Android phone as an input
./qcsuper.py --adb --wireshark-live

# Same, but dump to a PCAP file instead of opening Wireshark directly
./qcsuper.py --adb --pcap-dump /tmp/my_pcap.pcap

# Same, but using an USB modem exposing a Diag serial port
sudo ./qcsuper.py --usb-modem /dev/ttyHS2 --wireshark-live
```

Here is the current usage notice for QCSuper:

```
usage: qcsuper.py [-h] [--cli] [-v]
                  (--adb | --usb-modem TTY_DEV | --dlf-read DLF_FILE | -
                  [--info] [--pcap-dump PCAP_FILE] [--wireshark-live]
                  [--memory-dump OUTPUT_DIR] [--dlf-dump DLF_FILE]
                  [--json-geo-dump JSON_FILE] [--decoded-sibs-dump]
                  [--reassemble-sibs] [--decrypt-nas] [--include-ip-traf
                  [--start MEMORY_START] [--stop MEMORY_STOP]

A tool for communicating with the Qualcomm DIAG protocol (also called QC
DM).

optional arguments:
  -h, --help            show this help message and exit
  --cli                 Use a command prompt, allowing for interactive
                        completion of commands.
  -v, --verbose         Add output for each received or sent Diag packet

Input mode:
  Choose an one least input mode for DIAG data.

  --adb                 Use a rooted Android phone with USB debugging en
                        as input (requires adb).
  --usb-modem TTY_DEV   Use an USB modem exposing a DIAG pseudo-serial p
                        through USB.
  --dlf-read DLF_FILE   Read a DLF file generated by QCSuper or QXDM, en
                        interoperability with vendor software.
  --json-geo-read JSON_FILE
                        Read a JSON file generated using --json-geo-dump

Modules:
  Modules writing to a file will append when it already exists, and cons
  it Gzipped if their name contains ".gz".

  --info                Read generic information about the baseband devi
  --pcap-dump PCAP_FILE
                        Generate a PCAP file containing GSMTAP frames fo
                        2G/3G/4G, to be loaded using Wireshark.
  --wireshark-live      Same as --pcap-dump, but directly spawn a Wiresh
                        instance.
  --memory-dump OUTPUT_DIR
                        Dump the memory of the device (may not or partia
                        work with recent devices).
  --dlf-dump DLF_FILE   Generate a DLF file to be loaded using QCSuper o
```

```
                          QXDM, with network protocols logging.
  --json-geo-dump JSON_FILE
                          Generate a JSON file containing both raw log fra
                          and GPS coordinates, for further reprocessing. T
                          used in combination with --adb.
  --decoded-sibs-dump     Print decoded SIBs to stdout (experimental, requ
                          pycrate).

PCAP generation options:
  To be used along with --pcap-dump or --wireshark-live.

  --reassemble-sibs       Include reassembled UMTS SIBs as supplementary f
                          also embedded fragmented in RRC frames.
  --decrypt-nas           Include unencrypted LTE NAS as supplementary fra
                          also embedded ciphered in RRC frames.
  --include-ip-traffic    Include unframed IP traffic from the UE.

Memory dumping options:
  To be used along with --memory-dump.

  --start MEMORY_START    Offset at which to start to dump memory (hex num
                          by default 00000000.
  --stop MEMORY_STOP      Offset at which to stop to dump memory (hex numb
                          by default ffffffff.
```

Specifying `-` to pipe data from stdin or towards stdout is supported (gzipped cont

Using QCSuper with an USB modem

You can use QCSuper with an USB modem exposing a Diag port using the `--usb-m` `<device>` is the name of the pseudo-serial device on Linux (such as `/dev/ttyUSB0` possibilites) or of the COM port on Windows (such as `COM3`).

Please note that in most setups, you will need to run QCSuper as root in order to be handling serial port interference.

If you don't know which devices under `/dev` expose the Diag port, you may have to auto-detect it by stopping the ModemManager daemon (`sudo systemctl stop Mod` command: `sudo ModemManager --debug 2>&1 | grep -i 'port is QCDM-capable'`

Please note that if you're not able to use your device with for example ModemMana
is not totally setup and that it will not work neither with QCSuper. A few possible go

- You didn't apply the proper mode switching command for your device.
- If you bought a device that previously had a SIM from a different operator, your
  have to use the unlock code from the former operator and submit it to the devi
  `mmcli -i 0 --pin=<your_unlock_code>`

If your Qualcomm-based USB device doesn't expose a Diag port by default, you ma
the AT port in order to enable the Diag port:

```
AT$QCDMG
```

Please note that only one client may communicate with the Diag port at the same ti
instances, or QCSuper and ModemManager instances.

If ModemManager is active on your system, QCSuper will attempt to dynamically ad
the Diag port and restart its daemon, as it's currently the best way to achieve this. I

Supported devices

QCSuper was successfully tested with:

- Sony Xperia Z (Phone) - 4G - works out of the box after rooting an enabling ad
- Nexus 6P (Phone) - 4G - works out of the box after rooting an enabling adb
- ZTE MF823 (USB Modem) - 4G - may require to mode-switch the device to CD
  mode, then execute the AT command mentioned above
- ZTE MF667 (USB Modem) - 3G, 2011 - should work out of the box (may require
- Option Icon 225 (USB Modem) - 3G, 2008
- Novatel Ovation MC998D (USB Modem)
- ZTE WCDMA Technologies MSM MF110/MF627/MF636 (USB Modem)
- ZTE 403zt (USB Modem) - 4G
- OnePlus One and 3 (Phones)
- Andromax A16C3H (Phone)
- Samsung Galaxy S4 GT-I9505 (Phone)

Is it however aiming to be compatible with the widest possible range of devices bas
capture part.

Do no hesitate to report whether your device is successfully working or not through
issue.

Related tools using the Diag protocol

There are a few other open tools implementing bits of the Diag protocol, serving va

- ModemManager: the principal daemon enabling to use USB modems on Linux,
(labelled as QCDM) in order to retrieve basic information about USB modem de
- SnoopSnitch (specifically gsm-parser): chiefly an Android application whose pu
on the radio layer (IMSI catcher, fake BTS...). It also have a secondary feature t
PCAP, which does not provide exactly the same thing as QCSuper (LTE traffic i
example, device support may be different).
  - diag-parser: A Linux tool that derivates from the PCAP generation feature f
improved, designed to work with USB modems.
- MobileInsight: this Android application intends to parse all kinds of logs output
(not only those containing signalling information, but also proprietary debuggin
a specific XML representation format. Does not provide user-facing PCAPs (bu
backend for converting certain protocol information to XML).
- qcombbdbg: A debugger for the Qualcomm baseband setting up itself by hook
the Diag command that allows to write to memory, for the Option Icon 225 USB
- OpenPST: A set of tools related to Qualcomm devices, including a GUI utility all
the tiny embedded filesystem accessible through Diag (EFS).
- SCAT: A tool with similar GSMTAP generation abilities, taking as input a serial p
Exynos.